

Procesamiento de texto en **Python**

Diplomatura en Machine Learning



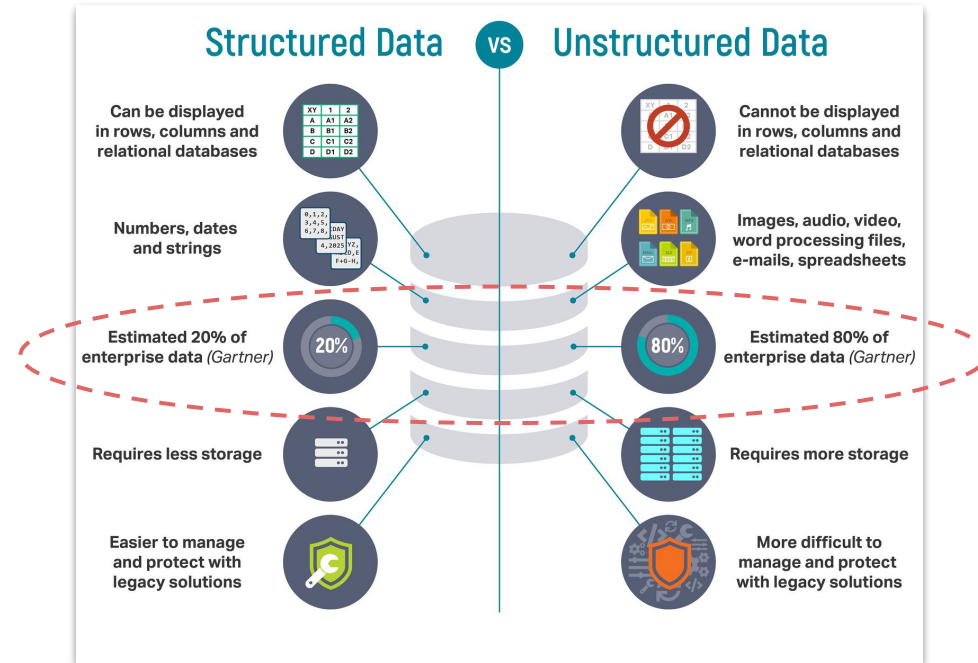
Aplicaciones del Procesamiento de Texto

- Traducción Asistida/Automática
- Reconocimiento de habla
- Corrección ortográfica
- Detección de tópicos
- Generación automática de resúmenes
- Extracción de Información
- Chatbot
- Sentiment Analysis



Tres tipos de presentación de los datos:

- **Datos estructurados:** Los datos se encuentran en formularios estandarizados o tablas. Son archivos de tipo texto que se suelen mostrar en filas y columnas con títulos. Son datos que pueden ser ordenados y procesados fácilmente por todas las herramientas de minería de datos.
- **Datos semi estructurados:** no presentan una estructura tan clara como la de las tablas, pero tienen cierta organización semántica. Los ejemplos de este tipo de datos son los archivos JSON y los XML.
- **Datos no estructurados:** Los datos se encuentran dispersos, no presentan una lógica de ordenamiento en común (Ej., Imágenes, Videos, Sonidos, Comentarios de los usuarios, Textos)



Lenguaje Natural vs. Lenguaje Formal

- **Lenguaje natural:** Cualquier lenguaje hablado que evolucionó de forma natural. Español, inglés, etc...
- **Lenguaje formal:** Cualquier lenguaje diseñado por humanos que tiene un propósito específico, como la representación de ideas matemáticas o programas de computadoras; todos los lenguajes de programación son lenguajes formales.

Pasos para su aplicación

Obtener la información

Consolidar un corpus a partir de datos que pueden provenir de diferentes fuentes.

Clase 2

Preparación de los datos

Eliminación de ruido, segmentación, normalización y reducción de léxico del texto.

Clase 3

Vectorización

Mapeo de tokens de un conjunto de datos a un vector correspondiente de números reales

Modelado

Aplicación de algoritmos para obtener inferencias y extraer información de los textos.

Clase 4

Análisis

Análisis, interpretación y navegación de los resultados

Cronograma de clases

Unidad	Tema	Fecha
Introducción	Conceptos básicos de Minería de Texto	21/10/2022
Creación del corpus	Manejo de archivos locales Web Scraping / APIs	28/10/2022
Preprocesamiento		04/11/2022
Clasificación Supervisada		11/11/2022

Aprendizaje automático



El aprendizaje automático (Machine Learning) consiste en una serie de técnicas que permiten a los sistemas informáticos predecir, clasificar, ordenar, tomar decisiones y, en general, extraer conocimientos de los datos **sin necesidad de definir explícitamente las reglas** para realizar esas tareas.

Es un subcampo de la inteligencia artificial que tiene como objetivo lograr que las **computadoras realicen tareas basadas en ejemplos sin programación explícita.**

Debido al continuo crecimiento de la complejidad de los negocios y de la cantidad de datos a analizar, se pasa de una inteligencia basada en reglas a una basada en datos.

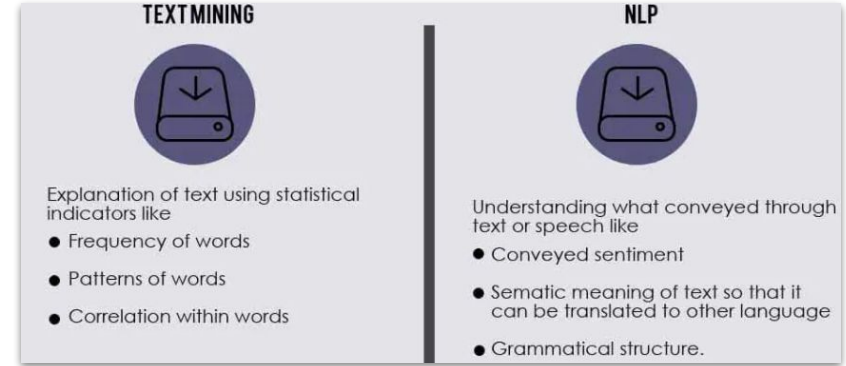
¿Minería de Textos o PLN?

Minería de Texto (Text Mining):

- Extraer información
- Encontrar patrones
- Se trabaja con reglas

Procesamiento de Lenguaje Natural:

- Entender el significado de un mensaje en lenguaje natural a través de la computadora
- Implica comprender la estructura gramatical del texto
- Importa el contexto



Introducción a la minería de textos en Python

```
31 del __init__(self, path):
32     self.file = None
33     self.fingerprints = set()
34     self.logdups = True
35     self.debug = debug
36     self.logger = logging.getLogger(__name__)
37     if path:
38         self.file.seek(0)
39         self.fingerprints.update(requests.get(path).text)
40
41 @classmethod
42 def from_settings(cls, settings):
43     debug = settings.getbool('SUPERFUTUR_DEBUG')
44     return cls(job_dir(settings), debug)
45
46 def request_seen(self, request):
47     fp = self.request_fingerprint(request)
48     if fp in self.fingerprints:
49         return True
50     self.fingerprints.add(fp)
51     if self.file:
52         self.file.write(fp + os.linesep)
```

¿Por qué Python?

Python: Lenguaje de alto nivel.

Lenguaje de alto nivel: Lenguaje diseñado para ser fácil de leer y escribir para los usuarios. La computadora debe traducirlo a un lenguaje de bajo nivel para entenderlo.

Lenguaje de nivel medio: Utilizan estructuras típicas de los lenguajes de alto nivel pero, a su vez, permiten un control a muy bajo nivel. Ej: C.

Lenguaje de bajo nivel: Lenguaje diseñado para ser fácil de ejecutar para una computadora. Ej: Código binario.

Popular: Actualización constante de las librerías, y comunidad activa de usuarios

Portabilidad: La cualidad de un programa que le permite ser ejecutado en más de un tipo de computadora.



Pandas

Una de las mejores opciones para trabajar con datos tabulares en Python es usar la Python Data Analysis Library (alias Pandas).

Permite:

- Leer y escribir datos en diferentes formatos: CSV, Microsoft Excel, bases SQL, etc.
- Seleccionar y filtrar de manera sencilla tablas de datos en función de posición, valor o etiquetas.
- Fusionar y unir datos (Igual al join en SQL)
- Transformar datos aplicando funciones tanto en global como por ventanas
- Manipulación de series temporales.

```
[ ] import pandas as pd
```

Pandas



DataFrame

	A	B	C	D
1		Student	First Name	Score
2	0	60	Olivia	90
3	1	100	John	75
4	2	80	Laura	82
5	3	78	Ben	64
6	4	95	Kevin	45

Un DataFrame es la estructura de datos más usada en Pandas. Permite guardar datos de distintos tipos (como caracteres, enteros, valores de punto flotante, factores y más) en columnas.

Es similar a una hoja de cálculo o una tabla de SQL o el `data.frame` de R.

Un DataFrame siempre tiene un índice con inicio en 0. (El índice refiere a la posición de un elemento en la estructura de datos).

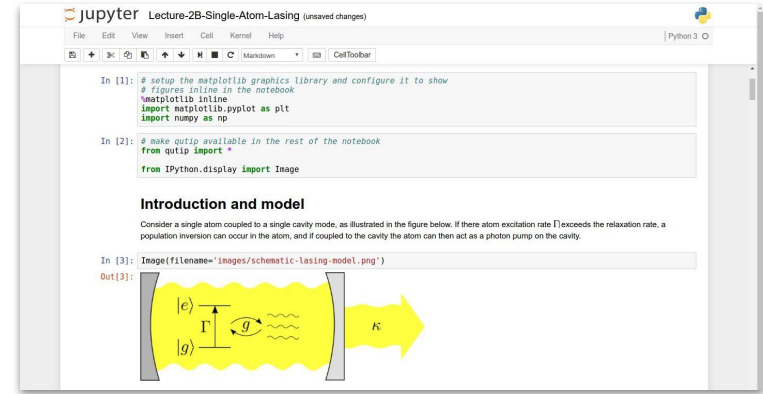


Jupyter Notebooks

Es entorno informático que promueve un diálogo entre el código y los textos explicativos.

Crea documentos que siguen una lista ordenada de celdas de entrada y de salida. Estas celdas pueden mostrar código, texto), fórmulas matemáticas, ecuaciones, o contenido multimedia (Rich Media).

Los documentos creados en Jupyter pueden exportarse a HTML, PDF, Markdown o Python.



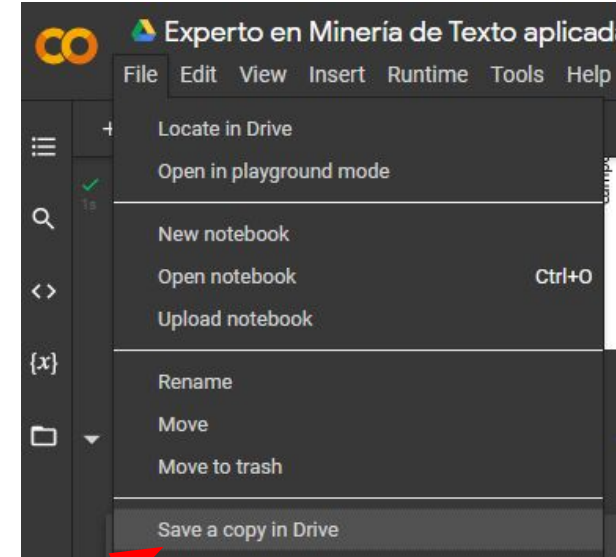
Google Colab

- En el curso vamos a trabajar con Notebooks montados en Google Colab, que es una herramienta gratuita que permite correr el código en la nube.
- De esta manera el código funcionará igual para todos/as sin importar el equipo con el que cuenten.
- *También está la posibilidad de descargar los archivos y correrlos en Anaconda, para lo cual hay que hacer click en File > Download > Download .ipnyb*



Consideraciones

- Para usar la plataforma es necesario contar con una cuenta de gmail / gsuite.
- **Los links a los códigos están restringidos para ser sólo lectura. Para poder modificarlos hay que hacer una copia, que se guardará en el Drive la cuenta desde la que se la crea.**



Librerías

NLTK

Es la librería más usada para Minería de Textos y Procesamiento del Lenguaje Natural en Python. Proporciona interfaces fáciles de usar a más de 50 corpus y recursos léxicos, junto con un conjunto de bibliotecas de procesamiento de texto para la clasificación, tokenización, el etiquetado, el análisis y el razonamiento semántico.

Spacy

Es una librería relativamente nueva que sobresale por su facilidad de uso y su velocidad a la hora de realizar el procesamiento de texto. Para trabajar con textos en español requiere descargar e instalar módulos especiales.

Freeling

Es una librería que cuenta con bastante desarrollo para el trabajo de textos en español. Tiene una curva de aprendizaje más intensa que NLTK, y su instalación no es sencilla (está originalmente escrita en C++). No se verá en el curso, pero se recomienda tenerla en cuenta si se desea profundizar en NLP.

Scikit-learn

Si bien no es específica de procesamiento de texto, es la biblioteca más utilizada para aprendizaje automático para Python, y usaremos los modelos Random Forest y Naive Bayes Classifier durante la clase de Clasificación automática de Textos.

Stanford CoreNLP

Paquete de PLN por la universidad de Stanford. Si bien es más popular en el lenguaje R, posee una interfaz con Python.

Cronograma de clases

Unidad	Tema	Fecha
Introducción	Conceptos básicos de Minería de Texto	21/10/2022
Creación del corpus	Web Scraping / APIs Manejo de archivos locales	28/10/2022
Preprocesamiento		04/11/2022
Clasificación Supervisada		11/11/2022

Creación de un corpus



Corpus

- El paso inicial de los trabajos con Minería de Textos es la creación y estructuración del corpus.
- Consiste en una colección de escritos compilados con la intención de que sean representativos y balanceados respecto de un fenómeno.
- Es por ello que el diseño de un protocolo de recolección y selección de los documentos que lo integran se trata de una etapa fundamental, puesto que implica un determinante del alcance de las conclusiones que se pueden proyectar a partir de su análisis.

Dataset

- **Conjunto de datos tabulados en cualquier sistema de almacenamiento de datos estructurados.**
- **Cada columna del Dataset representa una variable y cada fila corresponde a cualquier dato que estemos tratando**

Creación de un corpus



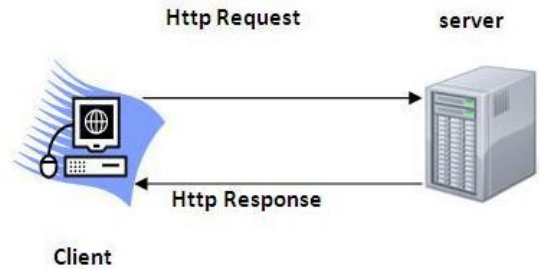
¿Qué es el Web Scraping?

El web scraping es la extracción automática de información de la web mediante el uso de bots o crawlers.

El web crawler o *spider* recorre e indexa las páginas web para el procesamiento de su contenido.



HTTP



- **HTTP** (Hypertext Transfer Protocol) es un protocolo que determina cómo las páginas web son transferidas desde un lugar a otro. Entre otras cosas, define el formato de los mensajes pasados entre los **clientes** HTTP (Navegadores, programas de líneas de comando...) y los **servidores** HTTP
- Como la web es un servicio, funciona mediante la combinación de clientes que hacen pedidos y servidores que reciben los pedidos.

El proceso consta de tres pasos principales

1)

Solicitar al servidor
donde está alojado
el sitio el código
HTML de la página
que nos interesa



2)

Recorrer el HTML
para obtener los
datos que buscamos
de esa página



3)

Consolidar los datos
en una tabla

HTTP Request

Librerías:
Requests
Selenium

HTML Parser

Librería:
Beautiful Soup

Data manipulation

Librería:
Pandas

Requests

- Dentro de las librerías disponibles para realizar interacciones HTTP la más usada es la librería Requests
- Posee una variedad de recursos importantes para la interacción con servicios web, relacionados con el encoding, la compresión, persistencia de conexión, seguridad y otros protocolos de comunicación.



HTTP Requests

De esta manera se hace un http request con la libreria Requests:

```
In [ ]: import requests  
        response = requests.get(URL)
```

- **URL** : Es la dirección url que estamos solicitando
- **requests**: Es la librería que importamos
- **request.get**: función de la librería para construir y enviar un GET requests

response : Es el nombre de la variable en donde asignamos el resultado del request, es un tipo de dato llamado **Response** y contiene información tanto de la comunicación, como del contenido solicitado al recurso remoto (o sea información que nos da el servidor).

Request ofrece la posibilidad de adquirir la información pero no de procesarla.

response.content



?



	operador	hora	cantidad
0	Q8	10:30	15
1	Q8	10:31	20
2	Q8	10:32	10
3	Q7	10:38	9
4	Q9	10:40	12
5	Q9	10:41	12

El proceso consta de tres pasos principales

1)

Solicitar al servidor
donde está alojado
el sitio el código
HTML de la página
que nos interesa



2)

Recorrer el HTML
para obtener los
datos que buscamos
de esa página



3)

Consolidar los datos
en una tabla

HTTP Request

Librerías:
Requests
Selenium

HTML Parser

Librería:
Beautiful Soup

Data manipulation

Librería:
Pandas

BeautifulSoup

Beautiful Soup permite obtener la información contenida en formato HTML/XML y recorrer estas estructuras de forma ordenada.

Esta librería transforma el contenido en objetos de python a través de un conjunto de herramientas para manipular el contenido, incluso si éste no se encuentra bien formado.

Los elementos que realizan la transformación desde el contenido hasta el objeto se conocen como **parsers**.



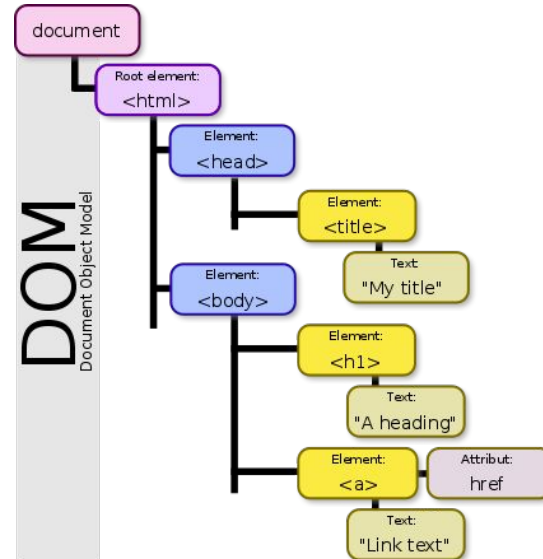
Document Object Model

El llamado a BeautifulSoup devuelve un objeto tipo DOM

response.content

```
b'\n<?xml version="1.0" encoding="utf-8" /><!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd"><html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en"><head><title>UserAgentString</title><meta name="description" content="Unknown user agent string. Find lists of user agents, bots, validators and other unknown user agent strings, use useragents, browser, crawler, spider, firefox, galeon" /><meta http-equiv="charset" content="iso-8859-1" /><link rel="stylesheet" href="useragentstring.css" /><script>
```

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(response.content, "html.parser")
```



```
from bs4 import BeautifulSoup
soup = BeautifulSoup(response.content, "html.parser")
```

Algunas funciones útiles:

- `.pretty()` : Devuelve una representación conveniente del elemento
- `.attrs` : Devuelve los atributos y valores de atributo del elemento
- `.find_all(tag, atributos)` : Busca todos los elementos pertenecientes a un tag, con algún atributo (si se especifica), no es obligatorio definir atributos.
- `.contents` : Lista solo los subelementos del nivel jerarquico inferior inmediato.
- `.descendants` : Lista todos los subelementos de los niveles inferiores.

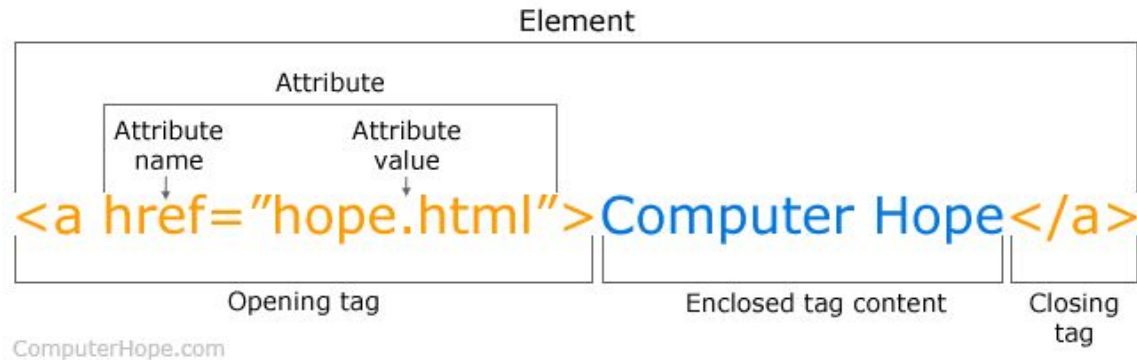
```
<tag atributo="valor"> Algun contenido </tag>
```

Estructura de HTML

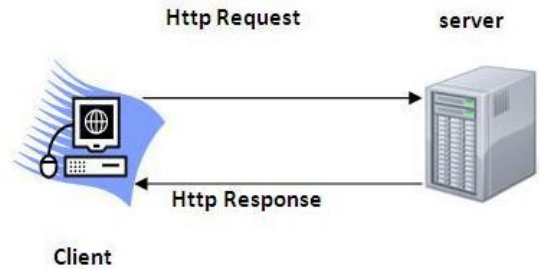
Se compone por elementos semánticos delimitados por etiquetas o *tags*, que se encapsulan en ángulos <>.

Los mismos pueden poseer atributos separados con espacio y con el valor entre comillas

Breakdown of an HTML Tag



HTTP



- **HTTP** (Hypertext Transfer Protocol) es un protocolo que determina cómo las páginas web son transferidas desde un lugar a otro. Entre otras cosas, define el formato de los mensajes pasados entre los **clientes** HTTP (Navegadores, programas de líneas de comando...) y los **servidores** HTTP
- Como la web es un servicio, funciona mediante la combinación de clientes que hacen pedidos y servidores que reciben los pedidos.

Requests

- Dentro de las librerías disponibles para realizar interacciones HTTP la más usada es la librería Requests
- Posee una variedad de recursos importantes para la interacción con servicios web, relacionados con el encoding, la compresión, persistencia de conexión, seguridad y otros protocolos de comunicación.



HTTP Requests

De esta manera se hace un http request con la libreria Requests:

```
In [ ]: import requests  
  
        response = requests.get(URL)
```

- **URL** : Es la dirección url que estamos solicitando
- **requests**: Es la librería que importamos
- **request.get**: función de la librería para construir y enviar un GET requests

response : Es el nombre de la variable en donde asignamos el resultado del request, es un tipo de dato llamado **Response** y contiene información tanto de la comunicación, como del contenido solicitado al recurso remoto (o sea información que nos da el servidor).

Las APIs

- Las APIs son un conjunto de métodos y formatos de datos muy claramente definidos, con el objeto de permitir la comunicación entre distintos componentes de software.
- En lo que respecta a Data Science, podemos ver a las API como una forma de hacer pedidos HTTP a un endpoint, para **enviar y recibir datos** estructurados, donde en vez de recibir páginas HTML, se reciben datos en una variedad de formatos, **JSON, XML, CSV**, etc.



REST es el tipo de arquitectura más común para pasaje de información desde y hacia endpoints

Consideraciones

- Muchas APIs no son accesibles de forma libre. Primero hay que registrarse como desarrollador para obtener una clave de autorización.
El objetivo detrás de solicitar la identificación y autorización de los clientes es evitar el acceso a datos restringidos, así como poder proteger el servidor de clientes abusivos que pudieran degradar el servicio para otros.
- Por ejemplo las APIs de Facebook sólo permiten obtener información propia del usuario que se ha identificado, pero no datos de otros. Al mismo tiempo establecen cuotas muy granulares para cada tipo de recurso, impidiendo la degradación del servicio por usuarios abusivos.

Formato JSON

- **JSON** es el acrónimo de JavaScript Object Notation. Es una forma de almacenar y transferir información de forma organizada y fácil de interpretar. Básicamente, nos da una colección de datos muy fácil de leer.
- JSON está construido por solo dos estructuras:
 - **Objetos**: Son colecciones de pares nombre/valor, en muchos lenguajes este tipo de colección se realiza mediante diccionarios, estructuras, tablas de hash, objetos, etc.
 - **Arrays**: Son una lista ordenada de valores. En muchos lenguajes esto es realizado mediante arreglos, vectores, listas, secuencias, etc.
- Estas son estructuras de datos **universales** ya que prácticamente todos los lenguajes de programación soportan este tipo de estructuras de alguna forma. Como JSON fue ideado como un formato de intercambio de datos universal, tiene sentido que se utilicen estas estructuras. Además de estas estructuras, JSON define otros tipos básicos: **string, número y booleanos**.

Formato JSON - *Estructura*

- **Objetos:** Un objeto es un conjunto no ordenado de pares nombre/valor, como en Python un diccionario. Un objeto comienza con { (llave izquierda) y termina con } (llave derecha). Cada nombre es seguido por : (dos puntos) y los pares nombre/valor son separados por , (coma).

La sintaxis es la siguiente:

```
{ nombre : valor, nombre : valor, ...}
```

ejemplo:

```
{"jugador": "Diego", "numero": 10, ...}
```

- **Arrays:** Un array es una colección ordenada de valores. Un array comienza con [y termina con] (corchetes). Entre ellos puede haber una cantidad arbitraria de valores separados por ,.

La sintaxis es la siguiente:

```
[ valor , valor, ...]
```

ejemplo:

```
[2, 3, 5, 7, 11, ...]
```

¿Qué son las Expresiones Regulares?

Las RegEx son una forma de describir patrones de texto. Nos permiten especificar tipos de cadenas que queremos buscar, extraer o modificar dentro de un texto.

Son un lenguaje para especificar la búsqueda de cadenas de texto.

```
! / <[>+]>  
TEST STRING  
<customer>  
  <reference>CD56381</reference>  
  <bill_to>  
    <contact>  
      <first_name>Christine</first_name>  
      <last_name>BAUMGARTNER</last_name>  
      <home_phone>4082845113</home_phone>  
      <email>MOMBAUM@AOL.COM</email>  
    </contact>  
    <address>  
      <line sequence="1">129 Bryan Ave.</line>  
      <city>San Jose</city>  
      <province_code>CA</province_code>  
      <province>CALIFORNIA</province>  
      <postal_code>95118</postal_code>  
      <country_code>US</country_code>  
    </address>  
  </bill_to>  
</customer>
```


¿Para qué se usan?

- En editores y procesadores de texto, pueden usarse para buscar y para reemplazar cadenas de texto.
- En programación se pueden usar para buscar segmentos de código (scrapping), validar inputs de texto, buscar y modificar nombres de archivo, etc.
- En el procesamiento de texto, se usan entre otras cosas para tareas de normalización de textos (como tokenización, stemming y lematización) y de extracción de entidades

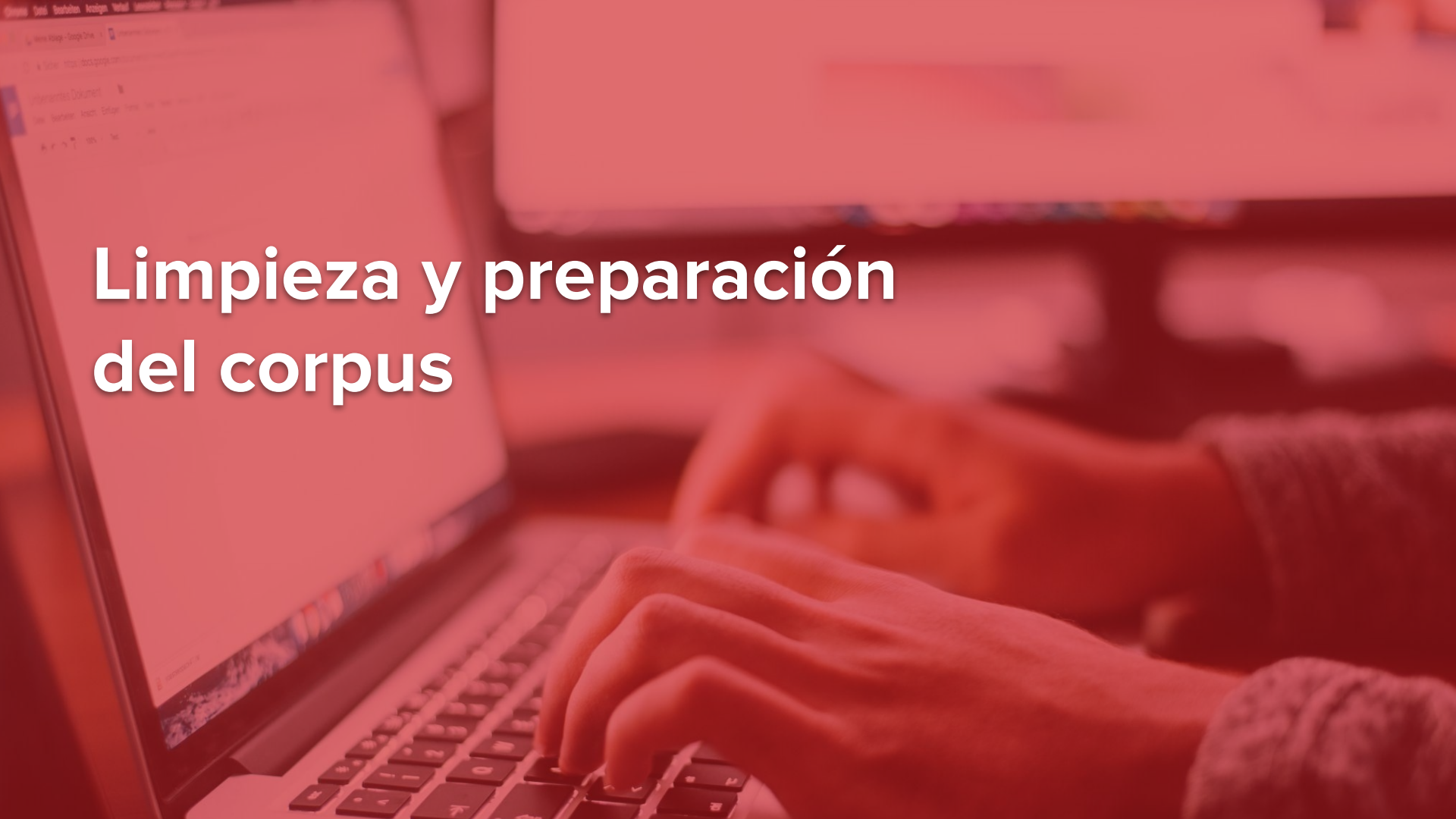
Motores de Expresiones Regulares

- Los motores de Expresiones Regulares son programas que permiten realizar búsquedas en el contenido de un texto. Cada lenguaje de programación tiene uno propio, y su funcionamiento puede variar.
- Si bien los meta caracteres y símbolos especiales suelen conservar su significado, la lógica de que tienen detrás puede variar y, en muchos casos, **una RegEx que funciona en Javascript no lo hará en Python, por ejemplo.**

La librería RE

En Python la librería para trabajar con RegEx se llama RE (no hace falta instalarla). Tiene cuatro funciones principales:

- `findall`: devuelve una lista con todos los matches.
- `search`: devuelve un objeto match
- `split`: permite cortar a la string con cada match.
- `sub`: reemplaza a los matches con otro valor.

A photograph of a person's hands typing on a laptop keyboard, overlaid with a semi-transparent red filter. The laptop screen is visible on the left, showing some text and a document icon. The background is blurred, showing another laptop and some papers.

Limpieza y preparación del corpus

Cronograma de clases

Unidad	Tema	Fecha
Introducción	Conceptos básicos de Minería de Texto	21/10/2022
Creación del corpus	Web Scraping / APIs	28/10/2022
	Manejo de archivos locales	
Preprocesamiento		04/11/2022
Clasificación Supervisada		11/11/2022

¿Qué es el Preprocesamiento?

El preprocesamiento del corpus consiste en la aplicación de las etapas necesarias para **transformar el texto con los que se esté trabajando, que se encuentran en lenguaje humano, a un formato legible para máquinas.**

Feature engineering

Se trata del proceso previo a la creación del modelo de predicción en el que se hace un **análisis, limpieza y estructuración de los campos de los datos**. Este proceso es uno de los más importantes y más costosos del proceso de predicción.

El objetivo es eliminar los campos que no sirven para hacer la predicción y organizarlos adecuadamente **para que el modelo no reciba información que no le es útil** y que podría provocar predicciones de poca calidad o confianza.

All Features



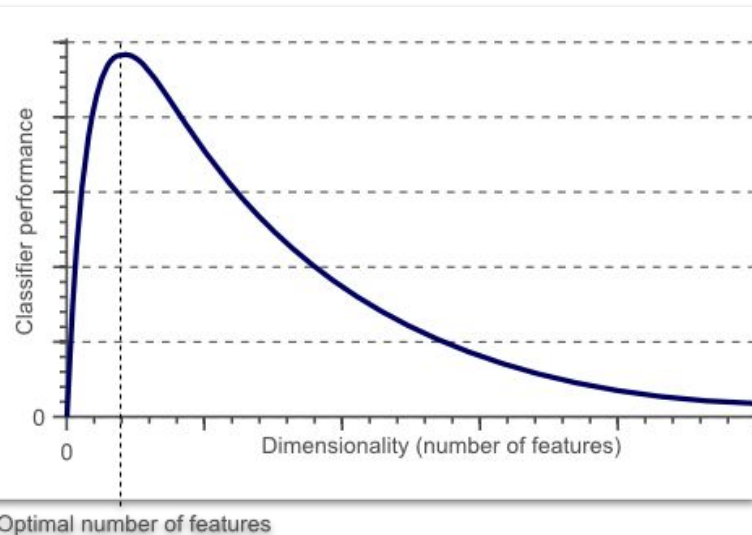
Feature Selection



Final Features



Maldición de la dimensionalidad



Aprender a partir de un conjunto de características de alta dimensionalidad normalmente requiere de una enorme cantidad de datos de entrenamiento para garantizar que haya varias muestras con cada combinación de valores posibles.

Existe un precio que los modelos pagan por tener más predictores a los cuales prestar atención.

Una forma de reducir la dimensionalidad de las características es usar solo un subconjunto de las mismas.

Pipeline

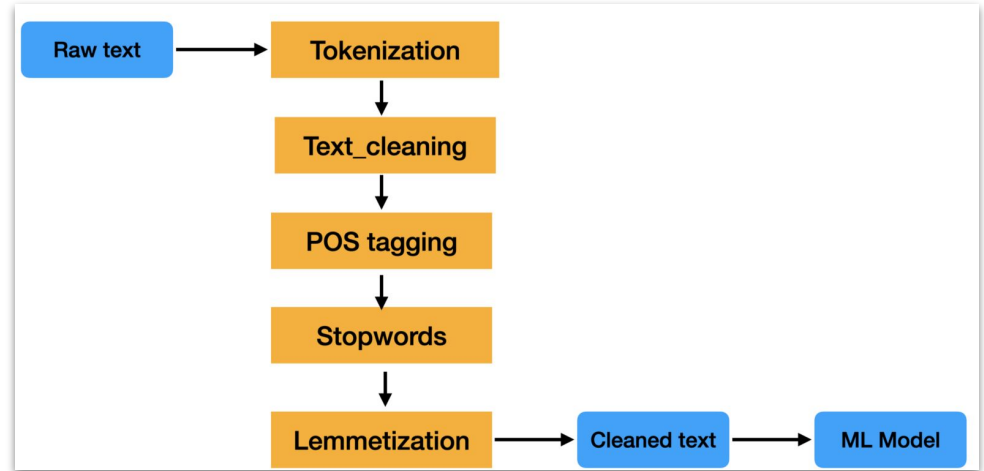


Un Pipeline es una serie de elementos de procesamiento de datos ordenados de tal modo que la salida de cada uno es la entrada del siguiente.

Es como si fuera una cadena de montaje pero en vez de orientada a la manufactura, orientada al procesamiento de datos e instrucciones.

Preprocesamiento de textos

- Tokenización
- Normalización
- Filtrado del ruido
- POS Tagging



IMPORTANTE

Es importante obtener un proceso que sea repetible y que produzca el mismo resultado independientemente del conjunto de datos que se le ingrese.

Esto es clave, ya que cuando llevemos nuestros modelos a producción y se encuentren con nuevas instancias de datos, deberemos de implementar las misma rutinas de preprocesameinto para que los datos sobre los que nuestro modelo fue entrenado y los datos que nuestro modelo ve en producción sean los mismos.

Segmentación

La **tokenización** divide el texto en entidades significativas:

- Palabras (espacios)
- Oraciones (puntuación)
- Caracteres (“”)
- N-gramas
 - Crea grupos de N (1, 2, 3+) entidades
 - “Data” y “Science” puede no implicar “Data Science”

N-gramas

Un n-grama es un subsecuencia de n-items. Ayudan a dar una idea más precisa del sentido de los términos, ya que una de las características que tiene el lenguaje es que el valor de una palabra depende del contexto en que a se encuentre.

La idea detrás de los taggers de n-gramas es que mirando las palabras previas a la palabra target podemos inferir con mayor precisión cuál es el tag adecuado que debemos asignar.

Ejemplo: Casa Rosada / Minería Texto

Normalización

La **normalización** consiste en eliminar:

- Mayúsculas / minúsculas
- Puntuación (ojo si se desean separar oraciones)
- Números (a veces)

Eliminación del ruido

Eliminar Stop Words

- Palabras comunes a todos los textos.
- La, el, los, de, a, en, sobre, con, por, que, tanto, etc....

Aplicar Stemming o Lematización

- Quedarse con la raíz de la palabra.
- Unifica tiempos verbales
- Agrupa palabras con mismo significado.

Son instancias sumamente útiles para reducir la dimensionalidad de los datos

POS Tagging - Corpus anotados

TAG	POS
ADJECTIVE	
AO	Adjective, Ordinal
AQ	Adjective, Qualifier
CONJUNCTION	
CC	Conjunction, Coordinate
CS	Conjunction, Subordinative
DETERMINER	
DA	Determiner, Article
DD	Determiner, Demonstrative
DE	Determiner, Exclamatory
DI	Determiner, Indefinite
DN	Determiner, Numeral
DP	Determiner, Possessive
DT	Determiner, Interrogative
PUNCTUATION	
Faa	Punctuation, ;
Fat	Punctuation, !
Fc	Punctuation, ,
Fd	Punctuation, ;
Fe	Punctuation, "
Fg	Punctuation, -
Fh	Punctuation, /
Fia	Punctuation, ¿
Fit	Punctuation, ?
Fp	Punctuation, .
Fpa	Punctuation, (
Fpt	Punctuation,)
Fs	Punctuation, ...
Fx	Punctuation, ;
Fz	Punctuation, other than those
INTERJECTION	
I	Interjection
NOUN	
NC	Noun, Common
NP	Noun, Proper
ADVERB	
RG	Adverb, General
RN	Adverb, Negative
ADPOSITON	
SP	Adposition, Preposition

TAG	POS
PRONOUN	
P0	Pronoun, Clitic
PD	Pronoun, Demonstrative
PE	Pronoun, Exclamatory
PI	Pronoun, Indefinite
PN	Pronoun, Numeral
PP	Pronoun, Personal
PR	Pronoun, Relative
PT	Pronoun, Interrogative
PX	Pronoun, Possessive
VERB	
VAG	Verb, Auxiliary, Gerund
VAI	Verb, Auxiliary, Indicative
VAM	Verb, Auxiliary, Imperative
VAN	Verb, Auxiliary, Infinitive
VAP	Verb, Auxiliary, Participle
VAS	Verb, Auxiliary, Subjunctive
VMG	Verb, Main, Gerund
VMI	Verb, Main, Indicative
VMN	Verb, Main, Infinitive
VMP	Verb, Main, Participle
VMS	Verb, Main, Subjunctive
VSG	Verb, Semi-Auxiliar, Gerund
VSI	Verb, Semi-Auxiliar, Indicative
VSM	Verb, Semi-Auxiliar, Imperative
VSN	Verb, Semi-Auxiliar, Infinitive
VSP	Verb, Semi-Auxiliar, Participle
VSS	Verb, Semi-Auxiliar, Subjunctive
DATE TIMES	
W	Date Times
UNKNOWN	
X	Unknown
ABBREVIATION	
Y	Abbreviation
NUMBERS	
Z	Figures
Zm	Currency
Zp	Percentage

El POS Tagging funciona a partir de **Corpus anotados** que contienen algún tipo de anotación lingüística (clase de palabra, sintaxis, entidades, valoración, etc.)

- En NLTK tenemos dos corpus con anotación de clase de palabra en español:

CoNLL2002 (Conference on Computational Natural Language Learning)

CessEsp (corpus multilingüe de español y catalán)

- En SpaCy el principal es:

es_core_news_sm: modelo multipropósito entrenado con textos de medios y redes sociales. Contiene componentes para realizar tokenización, POS tagging, parsers de dependencia y reconocer entidades (PER, LOC, ORG, MISC)

Stemming

Para poder avanzar en el procesamiento necesitaríamos que nuestro programa, además de distinguir tokens semánticamente relevantes de aquellos irrelevantes, pueda identificar por ejemplo todas las formas relacionadas con libros como pertenecientes a una misma clase

La librería que NLTK creó para habilitar su método de stemming se llama **Snowball**. Si bien las reglas fueron originalmente pensadas para el inglés, el sistema fue adaptado para otros idiomas.

NLTK trae una versión del Snowball Stemmer con reglas para varios lenguajes

changing
changed
change

stemming →

chang
chang
chang

studying
studies
study

stemming →

studi
studi
studi

REDUCIR A LA RAÍZ

Lematización

La lematización es un proceso lingüístico que consiste en, dada una forma flexionada (es decir, en plural, en femenino, conjugada, etc), hallar el lema correspondiente.

El lema es la forma que por convenio se acepta como representante de todas las formas flexionadas de un mismo término. **En la mayor parte de los casos corresponde con la forma de diccionario de palabra.**

```
[36] doc[215:240] #Texto original
, y todo el mundo se espantó al ver que los calderos, las pailas, las tenazas y los anafes se caían de su

lemmas[215:240] #Lemas
['',
 'y',
 'todo',
 'el',
 'mundo',
 'se',
 'espantar',
 'al',
 'ver',
 'que',
 'lo',
 'caldero',
 'los',
 'paila',
 'tenazas',
 'y',
 'lo',
 'anafe',
 'se',
 'caen',
 'de',
 'su']
```

REDUCIR A LA FORMA DE DICCIONARIO

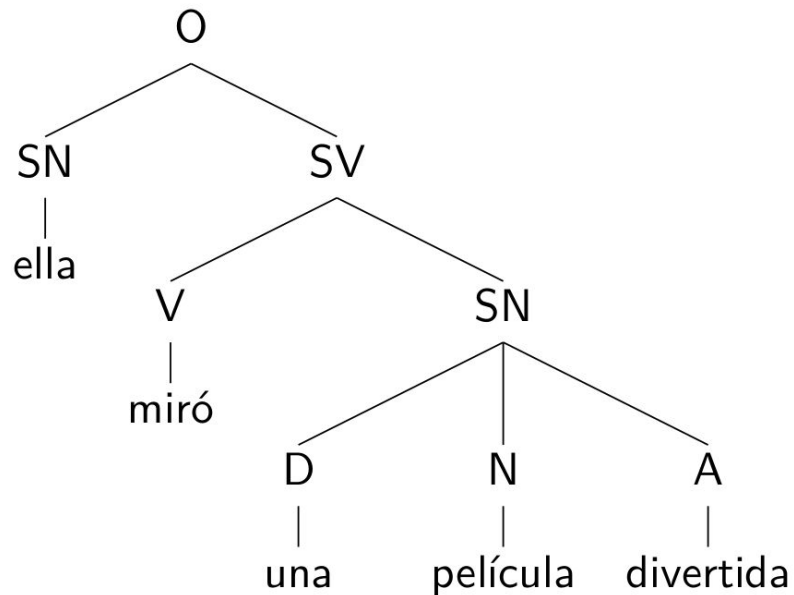
Parsers

Un parser es un algoritmo computacional que **recibe un *input*, lo analiza y lo devuelve ordenado de acuerdo a reglas** que conoce previamente, generalmente en forma de árbol. Tiene cargado un conjunto de reglas que le indican cómo dar formato al *input*. Suele ser parte de un proceso más complejo (lectura de archivos, compilador, etc).

Analizar una cadena de símbolos de acuerdo a una serie de reglas definidas previamente

En NLP:

- Recibe información no estructurada: una cadena (*string*) en lenguaje natural
- Consta de una gramática que contiene instrucciones de reescritura
- **Devuelve información estructurada: una representación formal de la estructura sintáctica**



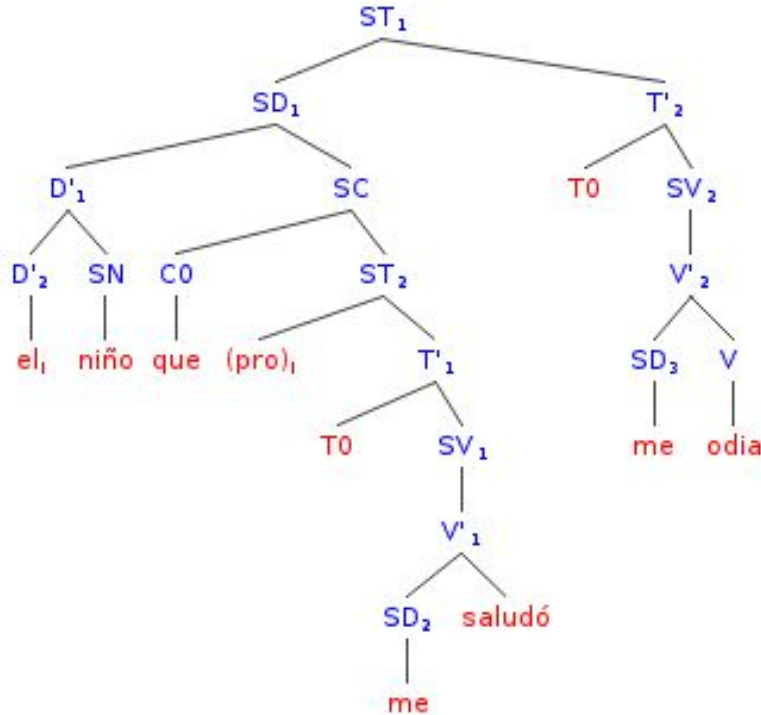
¿Por qué son necesarios los Parsers?

Una motivación para la creación y el uso de los parsers es principalmente el entendimiento.

Nos permite acceder a niveles más profundos del significado a partir del acceso a la estructura sintáctica de una oración.

Aunque sepamos la categoría de una palabra, es decir su POS, no tenemos información sobre las relaciones jerárquicas que se establecen entre ellas. El proceso de parsing nos devuelve esa información.

Constituyentes sintácticos



Un constituyente sintáctico es una palabra, o secuencia de palabras, que funciona en conjunto como una unidad dentro de la estructura jerárquica de una oración.

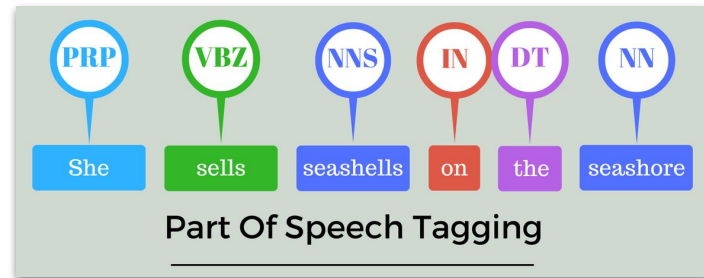
Un constituyente puede descomponerse frecuentemente en dos subsecuencias o más, cada una de las cuales es, a su vez, otro constituyente.

El conjunto de todos los constituyentes de una oración es un conjunto ordenado, en donde el orden se basa en la descomponibilidad de los constituyentes en subconstituyentes.

POS Tagging

¿Qué es taggear? ¿Para qué nos sirve?

- Asignar a cada palabra su clase gramatical ('part of speech').
- Convertir un objeto sentence (lista de palabras) en una lista de tuplas (word, tag).
- Paso previo necesario antes casi cualquier otra tarea de más alto nivel.



Clase gramatical

En el mundo de habla castellana, la gramática tradicional distingue nueve partes de la oración:

- **Sustantivos** designan cosas que tienen existencia; seres animados o inanimado, ideas...}
- **Pronombres** remplazan a los sustantivos y llevan a cabo las mismas funciones sintácticas que éstos.
- **Adjetivos** acompañan a un sustantivo y concretan alguna circunstancia cualidad o accidente de éste.
- **Verbos** expresan acciones, estados o procesos en un tiempo concreto
- **Adverbios** acompañan al verbo y lo caracterizan en algún aspecto
- **Preposiciones** su función es la de unir palabras con sus complementos
- **Conjunciones** nexos que relacionan palabras u oraciones entre sí
- **Interjecciones** expresan sentimientos de pena, sorpresa, ilusión
- **Determinantes** cuyo objetivo es concretar y delimitar el significado del sustantivo al que acompañan.

Cronograma de clases

Unidad	Tema	Fecha
Introducción	Conceptos básicos de Minería de Texto	21/10/2022
Creación del corpus	Web Scraping / APIs	28/10/2022
	Manejo de archivos locales	
Preprocesamiento		04/11/2022
Clasificación Supervisada		11/11/2022

Modelos para Vectorizar

Vectorización

La vectorización es el último paso del preprocesamiento, y consiste en el mapeo de tokens de un conjunto de datos a un vector correspondiente de números reales, que se utilizan como entrada para los modelos de Machine Learning **(recordemos que la mayor parte de los algoritmos requieren trabajar con valores numéricos).**

Transformar



Tokens en **Features**

Bag Of Words

El método más simple de vectorización
Cuenta términos que aparecen en los textos

Ejemplo:

- *John likes to watch movies. Mary likes movies too.*
- *John also likes to watch football games.*

```
"John", "likes", "to", "watch", "movies", "Mary", "likes", "movies", "too"
```

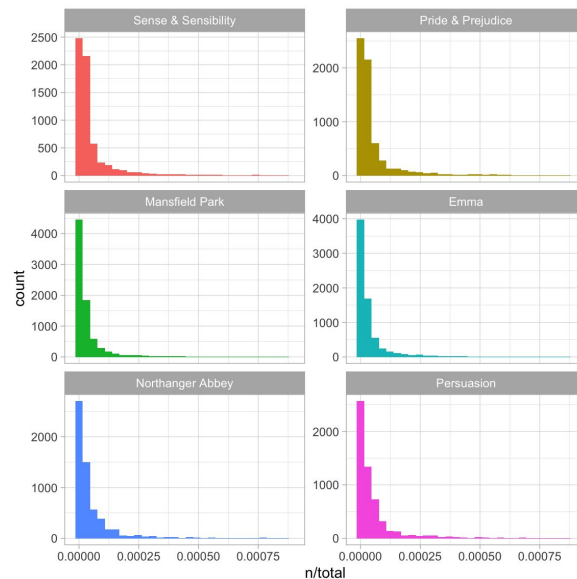
```
BoW1 =  
{ "John":1, "likes":2, "to":1, "watch":1, "movies":2, "Mary":1, "too":1 }
```

```
BoW2 =  
{ "John":1, "also":1, "likes":1, "to":1, "watch":1, "football":1, "games":1 }
```

Tf-Idf

Otorga un valor a cada término de acuerdo a su frecuencia en el documento dividida por una función de la cantidad total de documentos en los que aparece la palabra

La ventaja de este enfoque es que las palabras que aparecen en muchos documentos aumentan el denominador, y eso hace que su valor para cada documento baje, mientras que en cambio si una palabra aparece pocas veces en el corpus pero muchas en un único documento sube, ya que se asume que está proporcionando mucha información sobre él.



Ley de Zipf

TfIdf

$$w_{x,y} = \text{tf}_{x,y} \times \log \left(\frac{N}{\text{df}_x} \right)$$

TF-IDF

Term x within document y

$\text{tf}_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

Clasificación automática de textos



Aprendizaje supervisado

El aprendizaje supervisado es una técnica de aprendizaje automático cuyo objetivo es encontrar una función **a partir de datos de entrenamiento.**

Dependiendo de si es un problema de regresión o de clasificación, la función retornará un valor numérico o una etiqueta de clase (“label” en inglés).

Aprendizaje no supervisado

El aprendizaje no supervisado es un método de aprendizaje automático donde se busca generar un modelo de acuerdo a las observaciones. La principal diferencia con respecto al aprendizaje supervisado es que los datos no están etiquetados.

Para obtener el modelo, **la información de entrada se organiza y agrupa de acuerdo a ciertas similitudes o diferencias entre los datos.** De esta forma se pueden generar clases incluso cuando no fueron provistas anteriormente.

¿Qué es la clasificación automática?

El término ‘clasificador’ hace referencia a algoritmos utilizados para asignación de elementos a diferentes categorías.

Esta asignación de categorías es lograda mediante el “entrenamiento” del mismo a través de datos proporcionados, previamente clasificados.

Análisis de Sentimientos

El análisis de sentimientos es una rama de la clasificación de textos automatizadas que busca inferir la **actitud de un usuario** respecto a un tema en función de la **connotación positiva o negativa** del lenguaje ocupado en el documento

Clasificador bayesiano

Ventajas del clasificador Naive Bayes

- Veloz y Simple
- Sólo requiere una pequeña cantidad de datos de entrenamiento para estimar los parámetros
- Se usa como Baseline

Naive Bayes - Base

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

THE PROBABILITY OF "B" BEING TRUE GIVEN THAT "A" IS TRUE

THE PROBABILITY OF "A" BEING TRUE

THE PROBABILITY OF "A" BEING TRUE GIVEN THAT "B" IS TRUE

THE PROBABILITY OF "B" BEING TRUE

Aprendizaje supervisado

Implica predecir la probabilidad condicional de que un documento pertenezca a una clase $P(c_i|d_i)$ a partir de la probabilidad de los documentos dada la clase $P(d_i|c_i)$ y la probabilidad a priori de la clase en el conjunto de entrenamiento $P(c_i)$

La probabilidad de un documento dada la clase suele asumirse como la probabilidad conjunta de los términos que aparecen en el vector dada la clase

Estas probabilidades se obtienen a partir de los documentos de entrenamiento

$$P(c_i|d_j) = \frac{P(c_i)P(d_j|c_i)}{P(d_j)}$$

Naive Bayes - Simple y Multinomial

Clasificador Naive Bayes Simple

- Considera la probabilidad de aparición de cada término dada la clase de forma binaria, es decir si el término aparece o no.

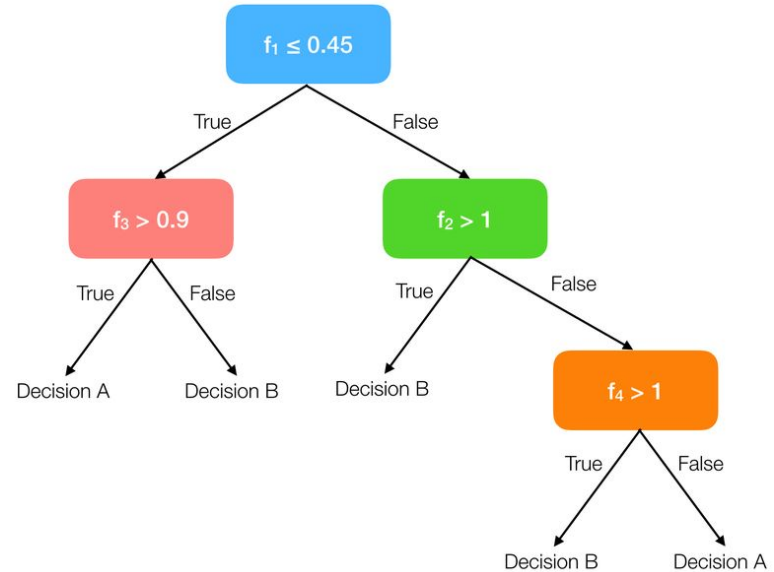
Clasificador Naive Bayes Multinomial

- Considera el número de apariciones del término para evaluar la contribución de la probabilidad condicional dada la clase

Random Forest

Clasificación con Árbol de Decisión

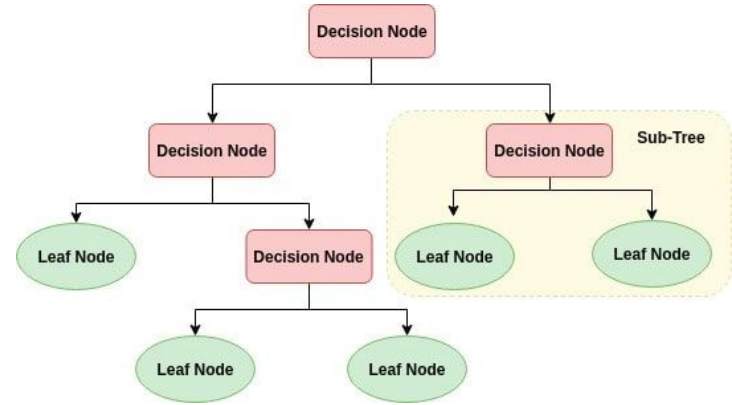
Un árbol de decisión es un modelo predictivo que divide el espacio de los predictores agrupando observaciones con valores similares para la variable respuesta o dependiente.



Funcionamiento

Los árboles de decisión clasifican los ejemplos clasificándolos por el árbol desde la raíz hasta algún nodo hoja, con el nodo hoja proporcionando la clasificación

Cada nodo en el árbol actúa como un caso de prueba para algún atributo, y cada borde que desciende de ese nodo corresponde a una de las posibles respuestas al caso de prueba. Este proceso es recursivo y se repite para cada subárbol enraizado en los nuevos nodos.

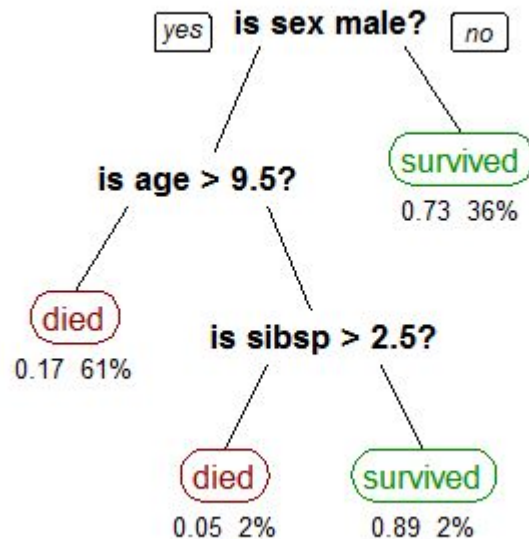


Estructura del árbol

Primer nodo o nodo raíz: en él se produce la primera división en función de la variable más importante.

Nodos internos o intermedios: tras la primera división encontramos estos nodos, que vuelven a dividir el conjunto de datos en función de las variables.

Nodos terminales u hojas: se ubican en la parte inferior del esquema y su función es indicar la clasificación definitiva.



Desventajas de los Árboles

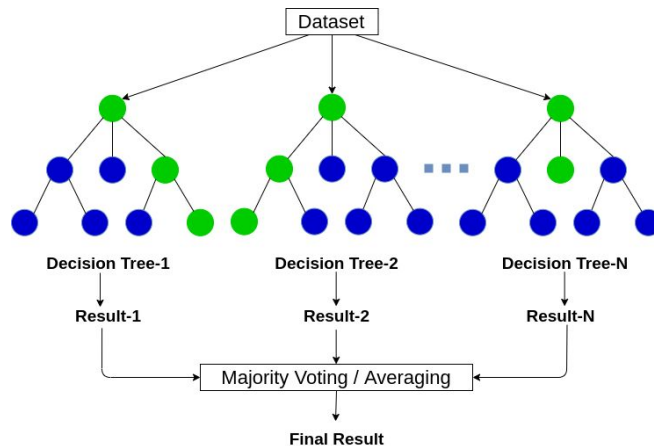
- **Sus resultados pueden variar mucho dependiendo de la muestra de datos usados para entrenar un modelo.**
- **Además es un modelo muy propenso al sobreajuste, esto es, hacerlos excelentes para clasificar datos que conocemos, pero deficientes para datos conocidos.**

Random Forest

Un Random Forest es un conjunto (ensemble) de árboles de decisión combinados con bagging.

Al usar bagging, lo que en realidad está pasando, es que distintos árboles ven distintas porciones de los datos. Ningún árbol ve todos los datos de entrenamiento. Esto hace que cada árbol se entrene con distintas muestras de datos para un mismo problema.

De esta forma, al combinar sus resultados, unos errores se compensan con otros y tenemos una predicción que generaliza mejor.



Conjuntos de datos

Hold-out

El método de hold-out validation básicamente separa el set de datos en 2 partes. Dado un data set de m registros, entonces el mismo se divide en t registros para entrenamiento y v registros para validación.



$$m = t + v$$

Validación de Modelos

Para evaluar modelos multiclase es usual generar métricas a partir de un conjunto de puntuaciones de predicción.

Accuracy es la suma de las predicciones correctas, sobre el total de predicciones. Un valor alto indica un alto porcentaje de clasificaciones correctas, aunque por sí solo no puede inferir que el modelo clasifique adecuadamente todas las clases.

Tuneo de hiperparámetros

Los hiperparámetros son parámetros que se configuran antes del entrenamiento del modelo y no forman parte del modelo como tal. Es decir que no son configuraciones que no se obtienen a partir de la data.

El método de Grid Search permite probar un subset de las combinaciones de hiperparámetros posibles.

